AI in CAD: Learner's Guide

Strategic Application of Artificial Intelligence in CAD · SGSITS Mechanical, B.Tech 2026-27 · Supervisor Dr. B. R. Rawal · theory, derivations and the real source code, one section per presenter

Problem statement. Turn a natural-language design requirement into a joint that is analytically sized, checked against every failure mode, modelled as an exact B-rep solid, drafted to SP 46 and cross-checked by FEA, with every number traceable to a formula and a source. The architecture is a **neuro-symbolic split**: a language model handles the one ambiguous step, parsing the intent, and deterministic symbolic code does everything that has a right answer.

> ***When faculty ask "how many epochs, what is your training data?"*** *Epochs and training loss measure how well a model learned a relationship from data. Our sizing has no learned weights, on purpose: the relationship (strength of materials) is already known exactly, so we encode it. The right metrics for an encoded model are* ***validation against known answers***, ***convergence of the iterative steps*** *and* ***error against an independent method****. Each section below gives those numbers in the same shape an ML project would: dataset, iterations, error. They're measured, not estimated.*

ML term	Our honest equivalent	Ours
Training data	The design-book equations and proportions (NPTEL IIT KGP, Bhandari, Unwin 1899, ISO tables)	6 joint models, 44 checks
Validation / test set	Published worked examples with printed answers, plus a golden set of English requests	Bhandari Ex 4.5, 4.2, Prob 4.1, NPTEL; 22 golden sentences
Epochs / iterations	The 1 mm grow-and-recheck loop (up to 400 steps) and FE mesh halving until the peak settles	knuckle 50 kN: 0 growth steps needed; FE flagged unless <5% move on halving
Test accuracy	Sizes and stresses matching the book exactly	Ex 4.5 7/7 sizes, 6/6 stresses
Generalisation	Any load or material, not just the trained range: it's the formula, so no extrapolation error	4000/4000 random loads (5-200 kN, 4 joints) pass every check
Regression tests	Automated pytest suite run on every change	301 cases

Architecture

Stage	Module	Deterministic?	Output
1. Intent extraction	aicad/parse.py	No (LLM) + guarded	Spec: joint, load, sizes, allowables
2. Analytical sizing	aicad/joints/<joint>.py :: design()	Yes	Design: dims, rules, raw sizes
3. Verification	<joint>.py :: _checks()	Yes	list[Check], all must pass
4. Solid modelling	build_parts() on CadQuery / OCCT	Yes	B-rep solids, STEP AP214
5. Drafting	aicad/freecad_pack.py (FreeCAD headless)	Yes	A3 first-angle sheet, DXF, FCStd
6. FE cross-check	aicad/fea.py + _fea_runner.py (gmsh, CalculiX)	Yes	von Mises peak, convergence flag
7. Material study	aicad/optimise.py :: compare()	Yes	mass and cost per material

Results to quote

Metric	Value
Joints, parts, checks	6 joints · 44 failure checks, all satisfied · busiest per joint 71-79%
Knuckle, 50 kN, governing	pin bending: 59.4 / 75 MPa = 79% utilisation
Knuckle sizes (mm)	D 35, D1 40, d 45, d0 90, d3 70, b 45, a 30
Validation vs textbook	Bhandari Ex 4.5: 7/7 sizes and 6/6 stresses exact · Ex 4.2: 9/10 · Problem 4.1 exact
FE validation	plain bar +1.2% (31.83 vs 32.22 MPa) · knuckle eye rod +11.3% (52.0 vs 57.9 MPa, September geometry)
Throughput	about 0.03 ms per complete knuckle design (1000-run average)
Test suite	301 pytest cases
Source errata found	NPTEL knuckle example fails its own eye-shear check by 23% · cotter spigot crushing omitted · bolt preload printed 284d kN, should be 2840d N

The core data model

Every joint returns the same record, so the UI, the report, the drawing and the tests treat all six joints identically. A `Check` passes only if the stress is within the allowable value. The tolerance 1e-9 absorbs floating-point error, and nothing else.

aicad\design.py :: Check

```
@dataclass
class Check:
    name: str
    stress_mpa: float
    allowable_mpa: float
    formula: str

    @property
    def passed(self) -> bool:
        return self.stress_mpa <= self.allowable_mpa + 1e-9
```

aicad\design.py :: next_preferred

```
def next_preferred(x: float, sizes: list[float]) -> float:
    for size in sizes:
        if size >= x - 1e-9:
            return float(size)
    raise ValueError(f"needs {x:.1f} mm, larger than the biggest size in
the table ({sizes[-1]} mm)")
```

aicad\design.py :: sizing_load

```
def sizing_load(value, design_factor):
    """Size for a bigger load than asked for, then report the stresses at
the real load.
    A factor of 1.25 leaves the governing check at about 80 % of the
allowed stress."""
    if value is None:
        return None
    return value * max(1.0, float(design_factor or 1.0))
```

The **design factor** is applied to the load, not to the stresses, so the allowables stay the book values. At 1.25 the governing check can't exceed 1/1.25 = 80% utilisation, and rounding up to preferred sizes pushes it below that. That's why every joint lands at 71-79%.

Atishay Jain

AI: language models, structured extraction and guarding against hallucination

Theory

How an LLM works

A transformer LLM maps text to **tokens** (sub-word units from byte-pair encoding, about 4 characters of English each) and is trained to maximise the likelihood of the next token, `p(x_t | x_<t)`. Self-attention lets every token weigh every other token, which is why it handles free phrasing like "a knuckle for five tonnes" well. Decoding samples from that distribution, and **temperature** controls how random the sampling is.

Why LLMs hallucinate, and why that matters here

The objective is *plausibility, not truth*. When the evidence is weak, the most likely continuation can still be a confident fabrication. LLMs also don't do arithmetic: "62,500 N" is a token pattern to them, not a computed value. In a design tool a hallucinated load is a **silent safety failure**, because the part looks fine and is sized for the wrong force.

Structured extraction

We constrain the model three ways: (1) a system prompt that defines the task, the units and "never invent a number"; (2) a JSON schema with an **enum** of the six joint names, so free text can't come back; (3) a post-hoc **provenance check** on every number. The first two reduce errors. Only the third *guarantees* no fabricated value gets through.

Why sizing is not machine learning

Sizing a joint is a closed-form inverse problem with an exact analytic answer, e.g. D = sqrt(4P/(pi sigma_t)). A regression model trained on past designs would only interpolate (it is wrong outside the training range), has no failure-mode semantics, and gives no reason a designer can sign off. ML earns its place where no analytic model exists. Here one does, so using ML would be strictly worse.

Text-to-CAD landscape

Generative text-to-CAD systems (commercial ones like Zoo, and research models trained on CAD sequence datasets such as DeepCAD) predict geometry directly. Their output is *shape-plausible* but not *load-verified*. Our pipeline inverts this: requirement → analysis → geometry, so the shape is a consequence of the engineering.

In our project

The system prompt (what the model is told):

aicad/parse.py lines 158-168

```
_SYSTEM = (
    "You extract a mechanical joint request into JSON. The request is in
English. "
    + _JOINT_MEANINGS +
    f"joint must be one of {JOINT_NAMES} or null. load_n is the axial load
in newtons "
    "(convert kN x1000, tonnes x9806.65, kgf x9.80665). rod_diameter_mm is
the rod diameter in mm. "
    "bolt_size is the metric bolt number (M12 -> 12). gibs is 1 or 2, only
for a gib and cotter "
    "joint, 2 when the request says double or two gibs. grip_mm is the
clamped thickness in mm. "
    "torque_nm is torque in newton metres, power_kw in kilowatts, speed_rpm
in revolutions per minute. "
    "tension_mpa, shear_mpa, crushing_mpa are allowable stresses in MPa. "
    "Use null for anything not stated. Never invent a number."
)
```

The guard. Every number the model returns must match a number literally present in the request, up to an allowed unit factor (x1000 for kN, x9.80665 for kgf, x9806.65 for tonnes), within 1%.

aicad\parse.py :: _traceable

```
def _traceable(value: float, text: str, factors=(1,)) -> bool:
    for n in _numbers_in(text):
        for f in factors:
            if abs(n * f - value) <= 0.01 * max(abs(value), 1e-9):
                return True
    return False
```

aicad/parse.py lines 220-224

```
        factors = _LOAD_FACTORS if key == "load_n" else (_PLAIN_FACTORS if
key in ("torque_nm", "power_kw") else (1,))
        if not _traceable(v, text, factors):
            spec.notes.append(f"AI gave {key}={v:g}, which is not in the
request; dropped")
            continue
        setattr(spec, key, int(v) if key in ("bolt_size", "gibs") else v)
```

Fallback chain. `parse()` tries each configured AI backend. The first one that answers is merged with the regex parse (the AI value wins, gaps are filled from the rules, disagreements are logged). If every backend fails (no key, no network, bad JSON), the regex parse alone is used, so the tool works offline.

aicad\parse.py :: parse

```
def parse(text: str, mode: str = "auto", runner=subprocess.run) -> Spec:
    """mode: 'rules' (offline), 'ai' (AI + rules fill-in, error if no AI
works),
    'auto' (AI + rules fill-in, else rules alone)."""
    rules = parse_rules(text)
    if mode == "rules":
        return rules
    errors = []
    for backend in ai_backends():
        try:
            spec = parse_ai(text, backend, runner=runner)
        except Exception as exc:  # network, quota, bad JSON, CLI missing:
try the next one
            errors.append(f"{backend}: {exc.__class__.__name__}: {exc}"
[:200])
            continue
        spec = merge(spec, rules)
        spec.notes = errors + spec.notes if errors else spec.notes
        return spec
    if mode == "ai":
        raise RuntimeError("no AI backend worked: " + ("; ".join(errors) or
"none configured"))
    rules.notes = [f"AI unavailable, used rules ({e})" for e in errors] or
rules.notes
    return rules
```

Evidence: our data, our iterations, our error

An ML project would report	What we report instead, measured
Pretrained model	Claude (Anthropic) used as-is. Zero fine-tuning epochs, by design: the parser needs language skill, not engineering knowledge
Test set	eval/golden_requests.json: 22 hand-labelled English requests, 12 fields each. An invented value counts as wrong
Accuracy, rules only	18/22 sentences fully correct, 0.0 s. Misses: 'two tonne', 'fifty kilonewton', 'half a tonne' (number words), 'pin joint with fork and eye' (synonym)
Accuracy, LLM	18/20 on the 18 Sep set (196 s for 20 calls). LLM + rules merged: 18/20. The LLM fixes phrasing, the rules fill gaps
Hallucination rate reaching the design	0 by construction: every LLM number must pass _traceable or it is dropped and logged
Code-generation eval	6 unlisted shapes: 5/6 correct on bounding box and volume, 1 failed after 2 attempts. That's why codegen is labelled experimental

Viva questions

1. **Is this a trained model?**
   No. It's a neuro-symbolic system. The LLM only parses intent. Sizing, checks and geometry are deterministic code from design books.
2. **Why not let the LLM do the calculation?**
   LLMs predict tokens and don't compute. They make arithmetic errors with full confidence, and nothing would show the mistake.
3. **Prompt says 'never invent a number'. Isn't that enough?**
   No. A prompt only lowers the probability. _traceable is a hard check: an untraceable number is dropped and logged.
4. **Can the guard reject a correct answer?**
   Yes, by design, e.g. 'double gibs' has no digit, so gibs is special-cased to {1, 2}. Rejecting a correct value just asks the user again. Accepting a wrong one would size a part wrongly.
5. **What is the JSON schema enum for?**
   It restricts joint to the six supported names or null, so the model can't return an unsupported joint.
6. **What happens with no internet?**
   parse() falls back to the regex parser. The notes record 'AI unavailable, used rules'.
7. **How is this different from text-to-CAD?**
   Text-to-CAD predicts shape. We derive shape from verified analysis, so every dimension has a formula and a source.

Bhavya Shrivastava
AI in mechanical engineering: PINNs, surrogates, generative design, digital twins

Theory

Physics-informed neural networks

A PINN approximates a field u(x, t) with a network u_theta and trains on a composite loss:

$$L(\theta) = (1/N)\,\Sigma\,|u_\theta(x_i) - u_i|^2 + \lambda \cdot (1/M)\,\Sigma\,|N[u_\theta](x_j)|^2 + \text{boundary terms}$$

The first term fits the data. The second is the **physics residual**: the governing PDE operator N (e.g. $\nabla\cdot\sigma + f = 0$ for static equilibrium) is evaluated by automatic differentiation at collocation points. Physics is a **soft constraint**, weighted by λ. With limited capacity or a badly tuned λ, the network can trade physics accuracy for data fit, so equilibrium is approximately but not exactly satisfied.

Surrogate models

A surrogate (Gaussian process, polynomial response surface, neural network) is trained on a design-of-experiments sample of an expensive simulation and then queried in microseconds. This makes optimisation and uncertainty studies affordable. The cost: it's only valid inside the sampled domain, and its error must be estimated (e.g. GP variance).

Generative design and topology optimisation

Topology optimisation (SIMP) gives each element a density ρ in [0, 1] with stiffness $E(\rho) = \rho^p E_0$ ($p \approx 3$) and minimises compliance subject to a volume fraction, using adjoint sensitivities. The output is a load-path-optimal material layout, which still needs interpretation for manufacturing (casting draft angles, machining access).

Digital twins

A digital twin is a model synchronised with a physical asset through sensor data. It is used for condition monitoring and remaining-useful-life prediction, and typically combines physics models with data-driven correction.

In our project

Our choice: physics as a hard constraint. In aicad the governing equations are not a penalty term. They are a *feasibility condition*: a design is only returned if every check passes, and a failing check deterministically grows the responsible dimension. In the knuckle, the eye diameter is grown 1 mm at a time until every eye and fork check passes:

aicad/joints/knuckle.py lines 94-107

```
    d2 = props["eye_outer_diameter_d2"]["times"] * d
    if P is not None:
        grown = False
        for _ in range(400):
            trial = dict(D=D, d=d, d2=d2, t=t_eye, t1=t_fork)
            if all(c.passed for c in _checks(Pd, trial, s) if
c.name.startswith(("eye", "fork"))):
                break
            d2 += 1
            grown = True
        if grown:
            notes.append(f"Eye and fork outer diameter grown to {d2:g} mm
because the bigger pin "
                         "leaves less metal around the hole.")

    dims = {
```

This is the correct tool for standard joints, where closed-form strength-of-materials models exist and are accepted by codes. A PINN or a surrogate would be the right tool for geometry with no closed form, e.g. a cast bracket, and our FE module is the natural source of training data for one.

A practical optimisation that is already in the project: `aicad/optimise.py` re-sizes the same joint for each material and compares mass and cost. At 50 kN, 30C8 gives 11.36 kg (about Rs 795), 40C8 7.63 kg, 40Cr1 4.05 kg, FG260 22.43 kg and aluminium 3.39 kg. Materials that are stronger in tension shrink the part, and cast iron's poor tensile strength makes it heaviest.

Evidence: our data, our iterations, our error

An ML project would report	What we report instead, measured
Constraint type	Hard: 44 analytic checks, a design is returned only if all pass. 4000/4000 random loads (5-200 kN) feasible
Optimisation dataset	5 materials compared at 50 kN: 30C8 11.36 kg, 40C8 7.63 kg, 40Cr1 4.05 kg, FG260 22.43 kg, Al 3.39 kg (mass range 6.6x)
Iterations	Deterministic grow-and-recheck in 1 mm steps (cap 400), instead of gradient descent on a penalty
Surrogate training data we already have	FE runs on 5 joints with convergence labels: a ready dataset if a stress-concentration surrogate is the next step
Speed vs simulation	0.03 ms per analytic design vs minutes per CalculiX run: over a million times faster for the checks that have closed forms

Viva questions

1. **Write the PINN loss.**
   Data misfit + λ · PDE residual (by autodiff at collocation points) + boundary-condition terms.
2. **Soft vs hard constraint?**
   Soft = a penalty that can be traded against data fit. Hard = a feasibility condition that must hold. aicad uses hard checks.
3. **Why not a PINN for the knuckle?**
   An exact closed-form model exists. A PINN would approximate a known answer, at higher cost and with no guarantee.
4. **Where does a surrogate fail?**
   Outside the sampled design space. Extrapolation error is unbounded.
5. **What is SIMP?**
   Topology optimisation with density-penalised stiffness $E = \rho^p E_0$. It minimises compliance at a fixed volume.
6. **How could AI extend this project?**
   Train a surrogate on our FE runs to predict stress concentration for unlisted shapes. Use topology optimisation for custom brackets.

Ansh Yadav

Machine design of joints: failure theory, derivations and Bhandari proportions

Theory

Allowable stresses and failure theories

For a ductile material, allowable tension σ_t = S_yt / FoS. Under the **maximum shear stress (Tresca)** theory, yield in shear happens at S_sy = 0.5 S_yt, so τ = 0.5 S_yt / FoS (von Mises gives 0.577 S_yt, which is the factor used for Bhandari Problem 4.1). Crushing (bearing) is a contact pressure on the projected area d x l, and it's allowed higher because it's local and compressive.

Knuckle joint: the governing equations

Rod tension: $P = (\pi/4)\, D^2\, \sigma_t \Rightarrow D = \sqrt{(4P / \pi\, \sigma_t)}$

Pin, double shear: $P = 2\,(\pi/4)\, d^2\, \tau \Rightarrow d = \sqrt{(2P / \pi\, \tau)}$

Pin bending derivation. The fork plates (thickness a) and the eye (thickness b) apply the load as distributed pressure along the pin. Model each fork reaction P/2 as uniform over a and the eye load P as uniform over b. Taking moments at mid-span:

$M = (P/2)(b/2 + a/3) - (P/2)(b/4) = (P/2)(a/3 + b/4) \quad \sigma_b = 32M / (\pi\, d^3)$

So the pin is sized for *both* shear and bending, and the larger one governs. Then the eye and fork are checked in tension and shear across (d0 − d) and in crushing on the projected area d x thickness.

Bhandari eq 4.26 proportions

d0 = 2d (eye and fork outer diameter) · a = 0.75D (fork) · b = 1.25D (eye) · d1 = 1.5d (pin head and collar) · D1 = 1.1D (enlarged rod) · x = 10 mm clearance.

Socket and spigot cotter

t = 0.31d · d3 = 1.5d · d4 = 2.4d · a = c = 0.75d · b = 1.6d · t1 = 0.45d · taper 1 in 32. The spigot diameter d2 and the socket diameter d1 are solved from their own net-section tension equations, e.g. $P = [(\pi/4)d2^2 - d2\ t]\ \sigma_t$, a quadratic in d2. The 1 in 32 taper is below the friction angle, so the cotter is self-locking.

In our project

The demo run (50 kN, σ_t 75, τ 60, σ_c 150 MPa, sized at 62.5 kN): rod 32.57 → D = 35 · pin shear 25.75 mm, pin bending 44.85 mm, so **bending governs** → d = 45 · d0 = 2d = 90 · b = 1.25D → 45 · a = 0.75D → 30. Busiest check: pin bending, 59.4/75 MPa = 79%.

Sizing code: note that b and a are rounded to standard sizes *before* the bending size is computed, the same order Bhandari uses in Ex 4.5:

aicad/joints/knuckle.py lines 66-92

```
    if rod_diameter_mm is not None:
        D = float(rod_diameter_mm)
    else:
        raw["rod_tension"] = math.sqrt(4 * Pd / (math.pi * s["tension"]))
        D = next_preferred(raw["rod_tension"], sizes)

    # the eye and fork thicknesses come from the ROD diameter (b = 1.25D, a
= 0.75D) and are rounded
    # up to the next standard size before anything is sized off them, as
Bhandari does in Ex 4.5
    # (37.5 -> 40, 22.5 -> 25); the pin is sized separately and raised when
shear or bending needs it
    t_eye, t_fork = next_preferred(k_t * D, sizes), next_preferred(k_t1 *
D, sizes)
    d = props["pin_diameter_d"]["times"] * D
    governing = "pin = rod diameter"
    if P is not None:
        raw["pin_shear"] = math.sqrt(2 * Pd / (math.pi * s["shear"]))
        if raw["pin_shear"] > d + 1e-9:
            d, governing = next_preferred(raw["pin_shear"], sizes), "pin
double shear"
        # sigma_b = 16 P (t1/3 + t/4) / (pi d^3), with t and t1 fixed by
the rod diameter
        raw["pin_bending"] = (16 * Pd * (t_fork / 3 + t_eye / 4) / (math.pi
* s["bending"])) ** (1 / 3)
        if bending_mode == "size" and raw["pin_bending"] > d + 1e-9:
            d, governing = next_preferred(raw["pin_bending"], sizes), "pin
bending"
            notes.append(
                f"Pin raised to {d:g} mm for bending (needs
{raw['pin_bending']:.1f} mm), the same step "
                "Bhandari Ex 4.5 and the NPTEL worked problem take. Set
pin_bending.mode to 'check_only' to keep the pin equal to the rod."
            )

    # the eye and fork outside diameter follows the PIN, d0 = 2d (Bhandari
eq 4.26j), so a pin
    # raised for bending carries its eye with it. If a check still fails,
grow it further: the
```

All nine checks, exactly as they run:

aicad\joints\knuckle.py :: _checks

```
def _checks(P, dims, s):
    D, dd, d2, t, t1 = dims["D"], dims["d"], dims["d2"], dims["t"],
dims["t1"]
    M = P / 2 * (t1 / 3 + t / 4)
    return [
        Check("rod tension", 4 * P / (math.pi * D**2), s["tension"], "4P /
(pi D^2)"),
        Check("pin double shear", P / (2 * math.pi / 4 * dd**2),
s["shear"], "P / (2 pi/4 d^2)"),
        Check("pin bending", 32 * M / (math.pi * dd**3), s["bending"], "32M
/ (pi d^3), M = P/2 (t1/3 + t/4)"),
        Check("eye tension", P / ((d2 - dd) * t), s["tension"], "P / ((d2 -
d) t)"),
        Check("eye shear", P / ((d2 - dd) * t), s["shear"], "P / ((d2 - d)
t)"),
        Check("eye crushing", P / (dd * t), s["crushing"], "P / (d t)"),
        Check("fork tension", P / ((d2 - dd) * 2 * t1), s["tension"], "P /
((d2 - d) 2 t1)"),
        Check("fork shear", P / ((d2 - dd) * 2 * t1), s["shear"], "P / ((d2
- d) 2 t1)"),
        Check("fork crushing", P / (dd * 2 * t1), s["crushing"], "P / (d 2
t1)"),
    ]
```

Evidence: our data, our iterations, our error

An ML project would report	What we report instead, measured
Reference dataset	Bhandari Ex 4.5 and 4.2, Problem 4.1, the NPTEL IIT KGP worked problems: printed answers as ground truth
Knuckle accuracy	Ex 4.5: 7/7 adopted sizes and 6/6 stresses (31.25, 25 N/mm2) exact. Problem 4.1: 14.47 and 13.47 mm exact
Cotter accuracy	Ex 4.2: 9/10 sizes. Spigot 37.92 and socket 52.04 mm solved from tension equations. Only b differs (the book uses FoS 4 on the cotter)
Errors found in the 'dataset'	NPTEL knuckle fails its own eye shear by 23%. Cotter spigot crushing omitted. Bolt preload 284d kN should be 2840d N
Coverage	44 checks over 6 joints. 103 design tests + 21 book-problem tests in pytest
Robustness	4000/4000 random loads, 5-200 kN, every check passes; busiest check stays at 71-79% of allowable

Viva questions

1. **Derive the pin bending moment.**
   Fork reactions P/2 uniform over a, eye load P uniform over b. Then M = (P/2)(a/3 + b/4) and σ = 32M/πd³.
2. **Why size the pin for bending as well as shear?**
   Shear alone needs 25.8 mm, bending needs 44.8 mm. A shear-only design would fail in bending.
3. **Why τ = 0.5 S_yt?**
   The maximum shear stress (Tresca) theory: yield when the max shear reaches half the tensile yield. It's conservative compared with von Mises (0.577).
4. **Why is the cotter self-locking?**
   A 1 in 32 taper (about 1.8°) is below the friction angle (about 8.5° at μ ≈ 0.15), so the axial load can't eject it.
5. **Why round b and a before sizing the pin?**
   Bending depends on a and b. Using the as-built (rounded) thicknesses makes the check consistent with the part actually made, as in Ex 4.5.
6. **Knuckle vs cotter: when to use which?**
   Knuckle: tension with angular freedom about the pin (valve rods, tie bars). Cotter: rigid axial joint, tension or compression.
7. **What is the NPTEL erratum?**
   Its worked example raises the pin for bending but checks the eye against the old pin, and fails eye shear by 23%. Our code grows d0 until the check passes.

Dharmendra Gupta

Engineering drawing and standards: SP 46, sectioning, limits and fits, GD&T

Theory

Projection

SP 46 (2003) / IS 10711 specify **first-angle projection**: the object sits in the first quadrant, between the observer and the plane, so the plan goes below the elevation and the left-side view goes to the right of it. The projection symbol (a truncated cone) goes in the title block.

Sections

The cutting plane is a thin chain line, thick at the ends and at changes of direction, labelled A-A, with arrows showing the viewing direction. Cut material is hatched with thin lines at 45°. Adjacent parts get opposite directions or different spacing. **Never sectioned lengthwise:** shafts, pins, bolts, nuts, keys, cotters, rivets, ribs and webs. Cutting a solid of revolution along its axis adds no information.

Lines and annotation

Continuous thick lines for visible outlines, thin dashed for hidden ones, thin chain for centre lines and axes of symmetry. Dimensions use aligned or unidirectional text, with no dimensions to hidden lines. A parts list (item, name, material, quantity) is linked to the views by balloons.

Limits and fits (IS 919 / ISO 286)

A fit is a hole tolerance zone plus a shaft one, e.g. **H7/g6** (location clearance), **H7/p6** (interference) or **H9/d9** (free running). The hole-basis system keeps the hole at H so standard reamers can be used. A knuckle pin needs a running clearance fit so the joint can articulate.

GD&T (ISO 1101 / ASME Y14.5)

The feature control frame gives the characteristic (form: flatness, cylindricity · orientation: perpendicularity, parallelism · location: position, concentricity · runout), the tolerance zone, and the datums A | B | C. Example: the pin bore ⊥ 0.02 to datum A (the eye face), so the pin doesn't bind across the fork.

In our project

What the drawing module produces: FreeCAD runs headless (`freecadcmd`) and builds TechDraw views of the STEP assembly on an A3 first-angle sheet: the front view as section A-A with hatching, the pin, cotter and bolts left unhatched, centre lines, dimensions, a title block and a parts list. It also writes a DXF for AutoCAD and the native FCStd. The same dims dictionary drives the solid and the drawing, so the drawing can't disagree with the calculation.

How the views are made: the module drives FreeCAD's TechDraw workbench from the dims and the parts list:

aicad/freecad_pack.py lines 223-232

```
        page = doc.addObject("TechDraw::DrawPage", "Sheet")
        tpl = doc.addObject("TechDraw::DrawSVGTemplate", "Template")
        tpl.Template = tpl_path
        page.Template = tpl
        texts = dict(tpl.EditableTexts)
        for key in list(texts):
            low = key.lower()
            if "title" in low and "sub" not in low:
                texts[key] = job["title"]
            elif "author" in low or "drawn" in low or "creator" in low:
```

Honest gap: limits and fits, surface finish symbols and GD&T are **not implemented yet**. Faculty raised this on 16 October. The next step is to attach H9/d9 to the pin and bore, and ⊥ to datum A on the eye.

Evidence: our data, our iterations, our error

An ML project would report	What we report instead, measured
Input	One dims dictionary per design feeds both the STEP solid and the TechDraw sheet
Outputs per run	A3 first-angle sheet (PDF), DXF, FCStd, STEP assembly: 4 formats from one source
Tests	3 FreeCAD pack tests (one per joint family, full sheet export) + a geometry test that every joint has drawing dimensions on real views
Standards applied	SP 46 first angle, section A-A, no lengthwise hatching of pins and cotters, centre lines, parts list
Known gap	No limits/fits or GD&T yet (16 Oct review). Next: H9/d9 on the pin, perpendicularity to datum A

Viva questions

1. **First vs third angle?**
   First angle (India, SP 46): plan below the elevation. Third angle (US): plan above it. The symbol is in the title block.
2. **Why aren't the pin and cotter hatched?**
   Solids of revolution and standard fasteners aren't sectioned lengthwise. Section A-A leaves them whole.
3. **What fit would you give the knuckle pin?**
   A running clearance fit, H9/d9, hole basis, so the joint can rotate freely.
4. **Read the fit H7/g6.**
   Hole: grade 7 tolerance with zero lower deviation (H). Shaft: grade 6, g sits slightly below zero. Together they give a small clearance location fit.
5. **What is a datum?**
   A theoretically exact reference feature (plane, axis) that GD&T tolerances are measured from.
6. **How do you guarantee the drawing matches the calculation?**
   Both are generated from the same dims dictionary. There's no manual redrafting step where they could diverge.

Ankur Dodke

FEA and CAD geometry: FEM formulation, meshing, convergence, B-rep and STEP

Theory

Finite element formulation

From the principle of virtual work, the displacement field inside each element is interpolated from nodal values, u = N u_e. Strain ε = B u_e, stress σ = D ε (D = the linear-elastic constitutive matrix from E and ν). The element stiffness is k_e = ∫ B^T D B dV, evaluated by Gauss quadrature. These are assembled into the global system

> K u = F (with displacement boundary conditions applied)

This is solved for u, then stresses are recovered at the integration points and reported as the **von Mises** equivalent σ_vm = √(½[(σ1−σ2)² + (σ2−σ3)² + (σ3−σ1)²]).

Meshing

gmsh generates tetrahedra from the B-rep. **Second-order (10-node) tets** capture bending and stress gradients far better than linear ones, which are over-stiff. The characteristic length sets the element size. Refine where the gradients are steep: the pin bore and the slot corners.

Convergence and singularities

The h-refinement test: halve the element size and check the peak stress stays put. A **stress singularity** (a sharp re-entrant corner, a point load, a rigidly fixed edge) gives a peak that grows without bound as the mesh refines. It's a mathematical artefact of the idealisation, not a real stress, and must not be reported as one.

B-rep and exchange

Boundary representation stores a solid as topology (shell → face → edge → vertex) bound to exact geometry (planes, cylinders, NURBS). OpenCASCADE is the open B-rep kernel, and CadQuery is a parametric Python API on top of it. **STEP AP214** (ISO 10303) carries exact B-rep plus assembly structure and colours between systems (NX, SolidWorks, Fusion). A mesh is lossy, B-rep isn't.

In our project

Load case. For the knuckle, the single-eye rod is held at the pin bore (where the pin bears) and pulled at the far rod end. The FE result is compared against the tool's own "rod tension" check:

aicad/fea.py lines 26-34

```
CASES = {
    "knuckle": {
        "part": "single_eye_rod",
        "fix": {"kind": "cylinder", "dia": "d"},          # the bore, where
the pin bears
        "load": {"kind": "plane_end", "axis": "x", "pick": "min"},   # the
far end of the rod
        "direction": [-1, 0, 0],
        "compare": "rod tension",
        "note": "Eye rod held at the pin bore, pulled at the rod end.",
    },
```

Validation. A 20 mm bar pulled by 10 kN: FE 32.22 vs hand 31.83 MPa, +1.2%. The residual is the edge effect at the fixed face. The knuckle eye rod: FE 57.9 vs hand 52.0 MPa, +11.3%, which is stress concentration at the enlarged-end transition that the uniaxial formula ignores. These were measured on the September geometry, and CalculiX runs only on the server.

Convergence guard: if the FE peak exceeds 1.5x the hand value, the mesh is halved and re-solved. If the peak moves by more than 5%, the result is flagged as not converged and isn't presented as a stress:

aicad/fea.py lines 124-137

```
    if r.get("ok") and check is not None and converge_check:
        ratio = r["max_von_mises"] / max(check.stress_mpa, 1e-9)
        if ratio > 1.5:
            job["mesh_mm"] = round(job["mesh_mm"] * 0.5, 2)
            job_file.write_text(json.dumps(job, indent=1))
            subprocess.run([exe, str(_SCRIPT)], capture_output=True,
text=True,
                           timeout=timeout, env=env)
            fine = json.loads(result_file.read_text())
            if fine.get("ok"):
                moved = 100 * (fine["max_von_mises"] - r["max_von_mises"])
/ r["max_von_mises"]
                r["fine_mesh_mpa"] = round(fine["max_von_mises"], 1)
                r["fine_mesh_nodes"] = fine["mesh_nodes"]
                r["peak_moved_pct"] = round(moved, 1)
                r["converged"] = abs(moved) <= 5
```

Geometry code: every part is built the way you would in NX: sketch, extrude, boolean. The eye rod is a disc, a rod and the D1 enlargement unioned together, with the pin bore cut through:

aicad/joints/knuckle.py lines 185-192

```
    eye_rod = (
        disc(d2, t, -t / 2)
        .union(rod_along_x(D, -(d2 / 2 + rod_len), d2 / 2 + rod_len))
        # the rod is enlarged to D1 where it meets the eye (Bhandari eq
4.26b)
        .union(rod_along_x(D1, -(d2 / 2 + D), d2 / 2 + D))
        .cut(hole_z(d, span))
        .clean()
    )
```

Evidence: our data, our iterations, our error

An ML project would report	What we report instead, measured
Solver validation	Plain bar 20 mm, 10 kN: FE 32.22 vs exact 31.83 MPa = +1.2%
Component result	Knuckle eye rod: FE 57.9 vs hand 52.0 MPa = +11.3% (September geometry)
Mesh-convergence study	Gib and cotter rod at 6.0, 4.0, 2.5, 1.6 mm mesh: peak 157, 180, 175, 191 MPa, wandering about 10% over 16x the nodes. Not settled, so flagged as a singularity, not reported as stress
Convergence rule	If peak > 1.5x hand, halve the mesh and re-solve. Converged only if the peak moves 5% or less
Geometry tests	48 pytest cases: each part is one valid solid, no two parts overlap (and the overlap test is proven able to fail), the knuckle eye can swing, the cotter sits in both slots, STEP keeps parts separate

Viva questions

1. **Write the element stiffness matrix.**
   k_e = ∫ B^T D B dV, with B the strain-displacement matrix and D the elasticity matrix, integrated by Gauss quadrature.
2. **Why 10-node tetrahedra?**
   Quadratic shape functions represent bending and stress gradients. Linear tets lock and come out over-stiff.
3. **Why validate on a plain bar first?**
   σ = P/A is exact there, so +1.2% isolates the solver and BC setup from any modelling question.
4. **Why is the eye rod 11% above hand?**
   The geometric stress concentration at the section change. The textbook formula assumes uniform stress.
5. **How do you detect a singularity?**
   The peak keeps rising under h-refinement instead of converging. Our code halves the mesh and flags >5% movement.
6. **B-rep vs mesh?**
   B-rep: exact topology plus geometry, editable, for CAD and STEP. Mesh: discrete approximation for analysis.
7. **Why STEP AP214?**
   ISO 10303 neutral exchange with assembly structure and colours. It opens natively in NX, SolidWorks and Fusion.